# 2.7. Inclusión de comentarios en el código (Criterio 2.g)

- **Resultado de Aprendizaje 2 (RA2):** *Escribe sentencias simples, aplicando la sintaxis del lenguaje y verificando su ejecución sobre navegadores Web.*

- **Criterio Curricular Oficial: CE 2.g** (*Se han añadido comentarios al código*).

- **Ponderación del Criterio:** (**2% del RA2** | **0,1% sobre la calificación final del módulo**).

### A. ¿Qué es un Comentario y cuál es su Función?

Un **comentario** es una porción de texto que el programador introduce en el archivo .js para documentar la arquitectura, advertir sobre restricciones o explicar algoritmos complejos.

Durante la fase de análisis léxico (*Tokenization/Parsing*), el motor de JavaScript descarta estas anotaciones por completo:

- **No generan código máquina:** No consumen memoria de ejecución en tiempo de ejecución.
- **No alteran el flujo:** No modifican condiciones, bucles ni llamadas a funciones.

### B. Tipos de Comentarios en JavaScript

#### 1. Comentarios de una Sola Línea (//)

Se inician con dos barras inclinadas (//). El analizador ignora todo el texto que aparezca a la derecha de las barras hasta llegar al salto de línea:

```
// Variable que controla el número de reintentos de conexión
let reintentos = 0;

let tasa = 0.21; // Comentario al final de una sentencia (inline)
```

#### 2. Comentarios Multilínea o de Bloque (/* ... */)

Se inician con los caracteres /* y finalizan con */. Permiten escribir párrafos completos de texto descriptivo repartidos en varias líneas:

```
/*
  Cálculo de la tarifa con recargo nocturno:
  Aplica un 15% sobre la base si la hora es superior a las 22:00.
  Cumple con la normativa de transporte metropolitano.
*/
function calcularTarifaNocturna(base, hora) {
  return hora >= 22 ? base * 1.15 : base;
}
```

**Trampa Sintáctica Clásica: Prohibición de Anidamiento Multilínea**
Un error frecuente en exámenes y prácticas de aula ocurre al intentar comentar temporalmente un bloque amplio de código que ya contiene otro comentario multilínea en su interior:

```
/*
  Inicio del bloque desactivado...
  /* Este comentario interior causará un fallo */
  let total = 100;
*/ // ERROR: El motor cerró el comentario en el primer "*/"
encontrado
```

El analizador léxico vincula el primer delimitador de apertura /* con el **primer cierre */ que localice**, dejando el texto posterior expuesto como código inválido, arrojando un Uncaught SyntaxError: Unexpected token.

**3. Comentarios Hashbang o Shebang (#!)**
Se introdujeron formalmente en ECMAScript 2023 para armonizar JavaScript con los sistemas Unix/Linux cuando los scripts se ejecutan directamente en la terminal sin invocar explícitamente el comando node:

```
#!/usr/bin/env node
// Obligatoriamente en la línea 1, columna 1 del archivo
console.log("Script CLI ejecutado directamente en el entorno del
sistema.");
```

**4. Comentarios de Marcadores de Tareas (TODO, FIXME, NOTE)**

En los IDEs modernos (Visual Studio Code, WebStorm), los comentarios de una sola línea suelen emplear etiquetas estandarizadas que resaltan visualmente y son indexadas por extensiones de control de calidad:

- `// TODO: Pendiente de implementar validación de email con expresión regular`
- `// FIXME: Error intermitente al parsear números decimales con coma`
- `// NOTE: Se mantiene esta función auxiliar por compatibilidad con navegadores antiguos`

**5. Comentarios de Licencia**

Cuando se inspeccionan librerías cliente distribuidas (como jQuery, Lodash o componentes empaquetados), el primer bloque multilínea empieza con un signo de exclamación (`/*! ... */`):

```
/*!
 * MiLibreria v1.0.0
```

```
 * (c) 2026 Autor - Licencia MIT
 */
```

- **Comportamiento en la minificación:** Los minificadores y empaquetadores (esbuild, Vite, Terser) reconocen `/*!` como un comentario legal protegido y **no lo eliminan** durante la compresión del script para producción.

### C. Documentación Formal con JSDoc

En el desarrollo profesional en cliente, los comentarios libres sin estructura son insuficientes. Se emplea el estándar **JSDoc**, una sintaxis basada en bloques que arrancan con una barra y dos asteriscos (/** ... */).

Visual Studio Code y los entornos modernos leen las etiquetas JSDoc para proporcionar autocompletado inteligente (*IntelliSense*), documentación flotante y comprobación estática de tipos en archivos JavaScript puros.

#### Principales Etiquetas del Estándar JSDoc:

- **@param {tipo} identificador - descripción**: Define el tipo de dato y propósito de un parámetro. Si se encierra entre corchetes [param], indica que es opcional.
- **@returns {tipo} descripción**: Detalla qué dato y de qué tipo devuelve la función.
- **@throws {TipoExcepcion}**: Documenta los errores que la función puede disparar deliberadamente.
- **@type {tipo}**: Documenta el tipo exacto de una constante o variable compleja.
- **@deprecated**: Marca una función o variable como obsoleta, advirtiendo al equipo de que será eliminada.

```
/**
 * Procesa la liquidación impositiva de una cesta de artículos.
 * @description Aplica el descuento por cupón y calcula la cuota
del IVA correspondiente.
 *
 * @param {number} baseImponible - Importe total antes de
impuestos (en euros).
 * @param {number} [tasaIVA=0.21] - Tipo de IVA aplicado en tanto
por uno (opcional).
 * @param {string} [codigoPromocional] - Cupón de descuento
aplicable.
 *
 * @returns {number} Importe final liquidado con impuestos.
 *
 * @throws {TypeError} Si la base imponible no es un dato de tipo
number.
 * @throws {RangeError} Si la base imponible es inferior o igual a
cero.
 */
```

```
function liquidarCesta(baseImponible, tasaIVA = 0.21,
codigoPromocional = "") {
  if (typeof baseImponible !== "number" ||
Number.isNaN(baseImponible)) {
    throw new TypeError("La base imponible debe ser estrictamente
numérica.");
  }
  if (baseImponible <= 0) {
    throw new RangeError("El importe base debe ser estrictamente
positivo.");
  }

  let baseFinal = baseImponible;
  if (codigoPromocional === "DESCUENTO10") {
    baseFinal *= 0.90;
  }

  return baseFinal * (1 + tasaIVA);
}
liquidarCesta(40);
```

**D. Directivas de Control para Herramientas de Calidad (ESLint / TypeScript)**

Existen comentarios funcionales con un formato especial que son interpretados por compiladores y linters para regular la severidad del análisis de código:

```
// 1. Activar comprobación de tipos de TypeScript en un archivo
.js nativo:
// @ts-check

// 2. Ignorar el análisis estático en la siguiente línea:
// @ts-ignore
let datoForzado = null;
datoForzado = 42;

// 3. Desactivar temporalmente una regla de ESLint sobre una línea
crítica:
// eslint-disable-next-line eqeqeq
if (datoForzado == 42) { // Permite comparación abstracta sin
advertencia del linter
  console.log("Coincidencia");
}
```

## TypeScript pero en código JS nativo

Si estuvieras programando en TypeScript, escribirías la variable así:

```
let edad: number = 25;
```

Con JSDoc, logras **exactamente el mismo nivel de seguridad** pero manteniendo tu archivo como JavaScript puro (`.js`):

```
// @ts-check

/** @type {number} */

let edad = 25;
```

¿Cómo funciona en la práctica?

Si habilitas la revisión en tu archivo (colocando `// @ts-check` en la primera línea) y agregas ese comentario, tu editor se comportará así:

**1. Te dará autocompletado inteligente (IntelliSense)**

Al saber que es un número, si escribes `miVariable.` el editor te sugerirá funciones exclusivas de números como `.toFixed()` o `.toString()`. No te mostrará funciones de texto como `.toUpperCase()`.

**2. Te alertará de errores antes de subir a producción**

Si intentas asignarle un valor que no sea un número, el editor te lo marcará con una línea roja inmediatamente:

```
// @ts-check

/**
 * @param {number} a
 * @param {number} b
 */
function sumar(a, b) {
    return a + b;
}

// El segundo argumento debe ser un número, no un texto.
const resultado = sumar(5, "10");

// JAVASCRIPT LO EJECUTA IGUAL:
console.log(resultado);
// Imprime: "510" (en lugar de 15, porque JS concatena los textos)
```

**Ejemplos de tipos más avanzados**

1. Variables con Tipos Básicos (Número, Texto, Booleano)

Le indicas al editor qué debe guardar la variable. Si intentas meter un tipo de dato incorrecto, el editor te avisará en tu pantalla.

```
// @ts-check

/** @type {number} */
let calificacionMinima = 6;

/** @type {string} */
let nombreProfesor = "Prof. Martínez";

/** @type {boolean} */
let elAlumnoAprobo = true;

// El editor marcará error aquí porque no es un número:
calificacionMinima = "seis";
```

2. Una Función Matemática (Parámetros y Retorno)

Documentas qué tipo de datos debe recibir la función entre los paréntesis y qué tipo de dato va a devolver (`@returns`).

```
// @ts-check

/**
 * Calcula el precio final sumándole el impuesto (IVA).
 * @param {number} precioBase - El costo del producto sin
impuestos.
 * @param {number} porcentajeIva - El porcentaje a aplicar (ej. 21
o 16).
 * @returns {number} El precio final con el impuesto incluido.
 */
function calcularPrecioFinal(precioBase, porcentajeIva) {
    let impuesto = (precioBase * porcentajeIva) / 100;
    return precioBase + impuesto;
}

// Al usar la función, el editor te recuerda qué poner:
let total = calcularPrecioFinal(100, 16);

// El editor te avisará que esto está mal porque le pasaste un
texto:
let totalConError = calcularPrecioFinal("cien", 16);
```

3. Crear una Plantilla de Objeto (`@typedef`)

Si en tus ejercicios manejas estructuras de datos fijas (como un alumno, un coche o un producto), puedes inventar tu propio tipo de dato personalizado y usarlo en cualquier variable.

```
// @ts-check

/**
 * @typedef {Object} Estudiante
 * @property {string} nombre - Nombre completo.
 * @property {number} edad - Edad en años.
 * @property {boolean} estaInscrito - Estatus del alumno.
 */

/** @type {Estudiante} */
const alumnoNuevo = {
    nombre: "Ana Gómez",
    edad: 20,
    estaInscrito: true
};

// Al escribir "alumnoNuevo." el editor te autocompletará: nombre,
edad, estaInscrito
console.log(alumnoNuevo.nombre);
```

4. Filtrar Listas (Arreglos de Objetos)

Puedes combinar tu tipo personalizado anterior para decirle al editor que una variable es una lista (`Array`) de esos objetos. Esto hace que ciclos como `forEach` o métodos como `.filter()` se vuelvan inteligentes.

```
// @ts-check

/**
 * @typedef {Object} Tarea
 * @property {string} titulo
 * @property {boolean} completada
 */

/** @type {Tarea[]} */
const misTareas = [
    { titulo: "Hacer ejercicio de JSDoc", completada: true },
    { titulo: "Estudiar para el examen", completada: false }
];

// Usamos el método .filter para separar las tareas pendientes
```

```
const pendientes = misTareas.filter(function(elemento) {
    // El editor sabe que 'elemento' es una Tarea, así que te
sugiere .completada
    return elemento.completada === false;

});
```

**E. El Impacto de los Comentarios en el Entorno Productivo: Proceso de Minificación**
Durante la fase de desarrollo, el código debe estar ampliamente documentado con JSDoc y comentarios arquitecturales para facilitar el trabajo en equipo y la mantenibilidad.

**En desarrollo:** Los comentarios aportan contexto indispensable para el equipo humano.

**En producción:** Sin embargo, cada carácter de comentario ocupa bytes en el archivo. Enviar un archivo con miles de líneas de comentarios al navegador ralentizaría la descarga por red del cliente. Para resolver esto, en producción se usan empaquetadores modernos para lograr tres objetivos:

1. **Minificación:** Quitan espacios en blanco, comentarios convencionales (// y /* ... */), comentarios de JSDoc (ya que el navegador no los necesita) y acortan los nombres de las variables para que el archivo pese un 80% menos.

   Únicamente se conservan aquellos comentarios de bloque que comienzan con un signo de exclamación /*! ... */, reservados legalmente para licencias de código abierto (MIT, Apache).

2. **Transpilación:** Traducen código moderno a una versión de JavaScript más antigua de forma automática, garantizando que tu app funcione incluso en celulares de hace 10 años.
3. **Inyección y optimización de CSS:** Procesan todas las clases que escribiste en tu HTML y archivos JavaScript para **generar un único archivo `.css` optimizado**. Este archivo final pasa por un proceso que elimina todos los estilos, selectores y reglas de diseño que escribiste pero que finalmente **no se están usando** en ninguna parte de la aplicación.

El estándar de la industria ha cambiado radicalmente. Herramientas antiguas y pesadas como **Webpack** han sido desplazadas por una nueva generación de herramientas que corren sobre lenguajes ultrarrápidos como Rust o Go.

Comparativa de los Empaquetadores Modernos

| Herramienta | Desarrollado en | Mayor Fortaleza | Ideal Para... |
| --- | --- | --- | --- |

| | | | |
|---|---|---|---|
| **Vite** | JavaScript / Go | El más popular, configuración cero y velocidad en desarrollo. | El 90% de los proyectos actuales (React, Vue, Vanilla JS). |
| **Turbopack** | Rust | Desarrollado por Vercel, optimizado para proyectos masivos. | Aplicaciones Full Stack gigantes con Next.js y React. |
| **Rspack** | Rust | Reemplazo directo y ultra veloz para proyectos viejos de Webpack. | Empresas grandes que quieren migrar de Webpack sin reescribir código. |
| **Bun** | Zig | Es un "todo en uno" (empaquetador, ejecutor y gestor de paquetes). | Proyectos modernos que buscan la máxima velocidad absoluta. |
| **Esbuild** | Go | El motor de bajo nivel más rápido del planeta. | Compilar librerías y potenciar las entrañas de herramientas como Vite. |

**F. Código autoexplicativo y ruido visual**

En el desarrollo de software profesional y en las revisiones de código corporativas (**Code Reviews**), el exceso de comentarios evidentes o redundantes se penaliza severamente y se considera **ruido visual** (*code smell*).

La filosofía del **Código Limpio (Clean Code)** dicta que el código debe ser **autoexplicativo**. Es decir, la semántica, la lógica y la intención del programa deben quedar claras mediante una excelente elección de nombres de variables, funciones y clases (por ejemplo, usar `intentosFallidosDeLogin` en lugar de una variable genérica `let x`).

Comentarios que explican *qué* hace una línea de código obvia:

```
// MAL: El código ya dice exactamente esto
let totalUsuarios = 0;
totalUsuarios++; // Incrementamos la variable en 1
```

Los comentarios deben reservarse exclusivamente para explicar el **¿por qué?** (decisiones, rendimiento, compatibilidad y no el *¿qué?*:

**JSDoc no entra en la categoría de "ruido visual" si se usa correctamente.**

- **Los comentarios comunes (`//`) dentro de las funciones:** Sí deben eliminarse casi por completo si la variable o la operación matemática es autoexplicativa.
- **Los bloques JSDoc (`/** ... */`):** No se consideran ruido porque su propósito no es explicar el código línea por línea, sino **definir reglas de datos (tipado)** para que el editor de texto (VS Code) funcione de forma inteligente en proyectos de JavaScript puro, o para generar documentación técnica automatizada.

## G. Estrategias de Comentarios Limpios (*Clean Code*) y Detección de *Code Smells*

### A. Casos de Refactorización: De Código Contaminado a Código Limpio

#### Caso 1: Comentarios Redundantes vs. Código Expresivo

```
// CASO INCORRECTO: El comentario no aporta nada que el lenguaje
no diga
// Comprueba si el empleado tiene más de 65 años
if (e.edad > 65) { ... }

// CASO CORRECTO: El código se documenta a sí mismo mediante
nombres semánticos
const esEdadJubilacion = empleado.edad >= 65;
if (esEdadJubilacion) { ... }
```

#### Caso 2: El Comentario "Disculpa" (*Apology Comment*)

```
// CASO INCORRECTO:

// Esta función es larguísima y un lío, pero básicamente calcula
el descuento del cliente:
function p(c, t) {
  // Multiplicamos por 0.85 si el cliente es tipo 1 y lleva más de
2 años...
  return c.t === 1 && c.antiguedad > 2 ? t * 0.85 : t;
}

// CASO CORRECTO: Extraer funciones atómicas con nombres legibles
function esClienteFidelizado(cliente) {
  const ES_PREMIUM = cliente.tipo === "VIP";
  const TIENE_ANTIGUEDAD = cliente.antiguedadEnAnios > 2;
  return ES_PREMIUM && TIENE_ANTIGUEDAD;
}
```

```
function aplicarDescuentoFidelidad(cliente, importeBase) {
  const FACTOR_DESCUENTO = 0.85;
  return esClienteFidelizado(cliente) ? importeBase *
FACTOR_DESCUENTO : importeBase;
}
```

**Caso 3: Cuándo un Comentario es Estratégicamente Necesario**

Un comentario es legítimo y necesario cuando explica **el motivo de una anomalía** que un revisor podría interpretar como un error (justificación de decisiones, parches por bugs en librerías, advertencia de consecuencias críticas, enlaces a tickets / normativas legales):

```
// ACEPTABLE Y NECESARIO: Explica una decisión técnica
contraintuitiva
function compararImportes(a, b) {
  /* BUG-4102: No utilizamos Math.abs(a - b) < Number.EPSILON
directamente debido a que las cantidades provienen serializadas de
una API bancaria externa con errores de redondeo de 3 decimales
fijos. Ref: https://normativa.banco.es/norma-redondeo-2024 */

  return Math.round(a * 100) === Math.round(b * 100);
}
```

**Caso 4: Eliminación de Código Muerto Comentado (*Zombie Code*)**

Dejar fragmentos antiguos de código comentados "por si acaso se necesitan luego" ensucia el archivo y confunde al equipo. Para conservar versiones anteriores existe el sistema de control de versiones (**Git**). La política estándar es borrar todo código no utilizado.

**H. Documentación Automática de APIs Front-End:** En empresas de desarrollo web, el código se documenta sistemáticamente con JSDoc no solo para apoyo en el editor, sino para generar portales de documentación web navegables de forma automática mediante herramientas de integración continua como **TypeDoc** o **JSDoc**.

**Generación Automatizada de Portales Web con JSDoc**

En el desarrollo de software industrial, la documentación escrita en los comentarios no se queda aislada dentro de los archivos de código fuente. Se procesa a través de herramientas de integración continua para generar sitios web estáticos de consulta técnica, permitiendo que cualquier miembro del equipo de desarrollo, auditoría o soporte explore la arquitectura de la aplicación, sus métodos, parámetros y tipos de retorno sin necesidad de abrir el código fuente.

Ejecuta el comando `jsdoc` seguido del nombre del archivo o la carpeta que quieres documentar:

npx `jsdoc index.js`

Ver el resultado:

Al ejecutar el comando, JSDoc creará automáticamente una carpeta llamada **`out/`** en tu directorio.

1. Entra en la carpeta `out/`.
2. Abre el archivo **`index.html`** en tu navegador.

## I. ACTIVIDADES

#### Ejercicio 1: Creación de Bloque JSDoc Profesional

**Enunciado:** Escribe la documentación formal completa en formato JSDoc para la siguiente función que valida el DNI español y genera la documentación.

```
function validarDNI(numeroDNI, letra) {
  const letras = "TRWAGMYFPDXBNJZSQVHLCKE";
  return letras[numeroDNI % 23] === letra.toUpperCase();
}
```